MLOps
(S1-25_AIMLCZG523)
ASSIGNMENT - I

Production-Readiness Documentation

Group No: 13

Assignment Group Members:

Name	BITS ID	Contribution
KOUSHIK SADHUKHAN	2024aa05797	100%
CHOCALINGAM L	2024ab05275	100%
GEETANJALI ANANTRAO UDGIRKAR	2024aa05800	100%
G PRANAVANATHAN	2024ab05277	100%
DURGA PRASAD YADAV	2024ab05147	100%

Production-Readiness Documentation

- This documents shows how the **Heart Disease MLOps Pipeline** satisfies production-readiness requirements.
    - All scripts must execute from a clean setup using the requirements file.
    - Model must serve correctly in an isolated environment (Docker; container build/test proof required).

        - Pipeline must fail on code or test errors and give clear logs.

- The evidence is derived directly from the codebase, Docker configuration, and CI/CD workflows.

        - **Code Base: https://github.com/2024ab05275-chocs/mlops-heart-disease/tree/main**
        - **Docker File: https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/main/Dockerfile**
        - **CI/CD Pipeline: https://github.com/2024ab05275-chocs/mlops-heart-disease/actions/runs/20704907045**
        - **Scripts for Local (Docker Desktop):**
            - **https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/scripts/setup/run_scripts.sh**
            - **Docker: https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/scripts/run_docker.sh**
            - **Kubernetes & Ingress: https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/scripts/run_local_k8s_ingress.sh**

1. Clean Setup & Dependency Reproducibility

Requirement

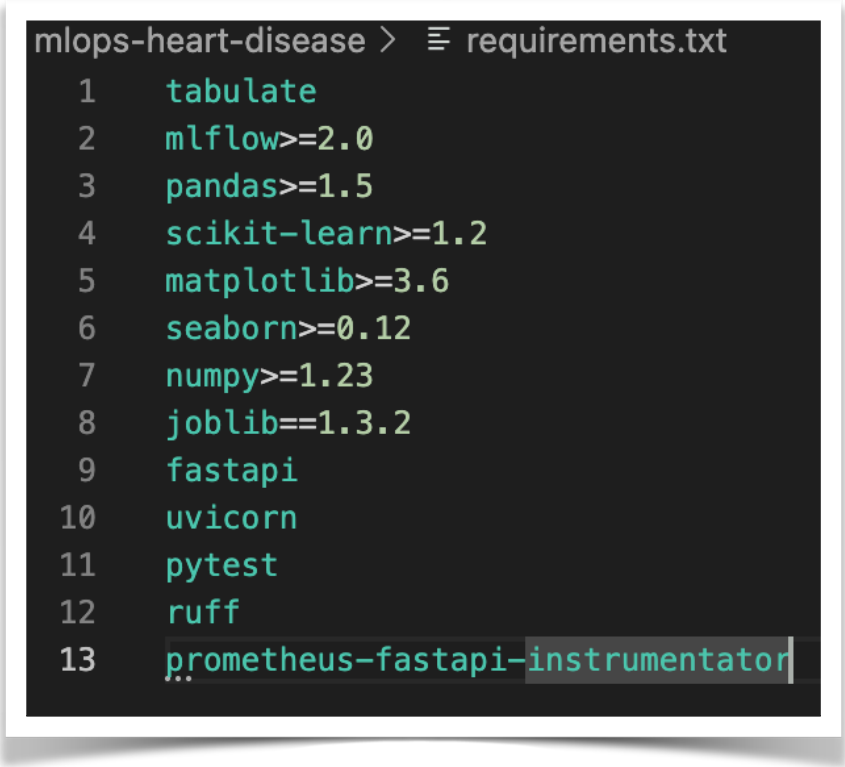
All scripts must execute from a clean setup using the requirements file.

Implementation

- The project provides a **single source of truth for dependencies** via `requirements.txt`.
- All Python scripts (training, evaluation, API serving, and tests) rely exclusively on these pinned dependencies.
- No global or system-level Python packages are assumed.

Evidence in Codebase

- **Requirements file (Dependencies):**
  - https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/requirements.txt
- **Setup Script:**
  - https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/scripts/setup/run_scripts.sh
- GitHub Actions workflow installs dependencies using:
  `pip install -r requirements.txt`



- **Docker image build also installs dependencies only from `requirements.txt`.**

Validation Steps (Local)

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pytest
Run https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/scripts/setup/run_scripts.sh
```

Successful execution from a fresh virtual environment demonstrates reproducibility and clean setup compliance.

2. Isolated Model Serving (Dockerized Deployment)

Requirement

Model must serve correctly in an isolated environment (Docker; container build/test proof required).

Implementation

- The project includes a **Dockerfile** that packages:
  - Application source code
  - Trained model artifacts
  - Runtime dependencies
- The container exposes the API service and runs independently of the host environment.

Evidence in Codebase

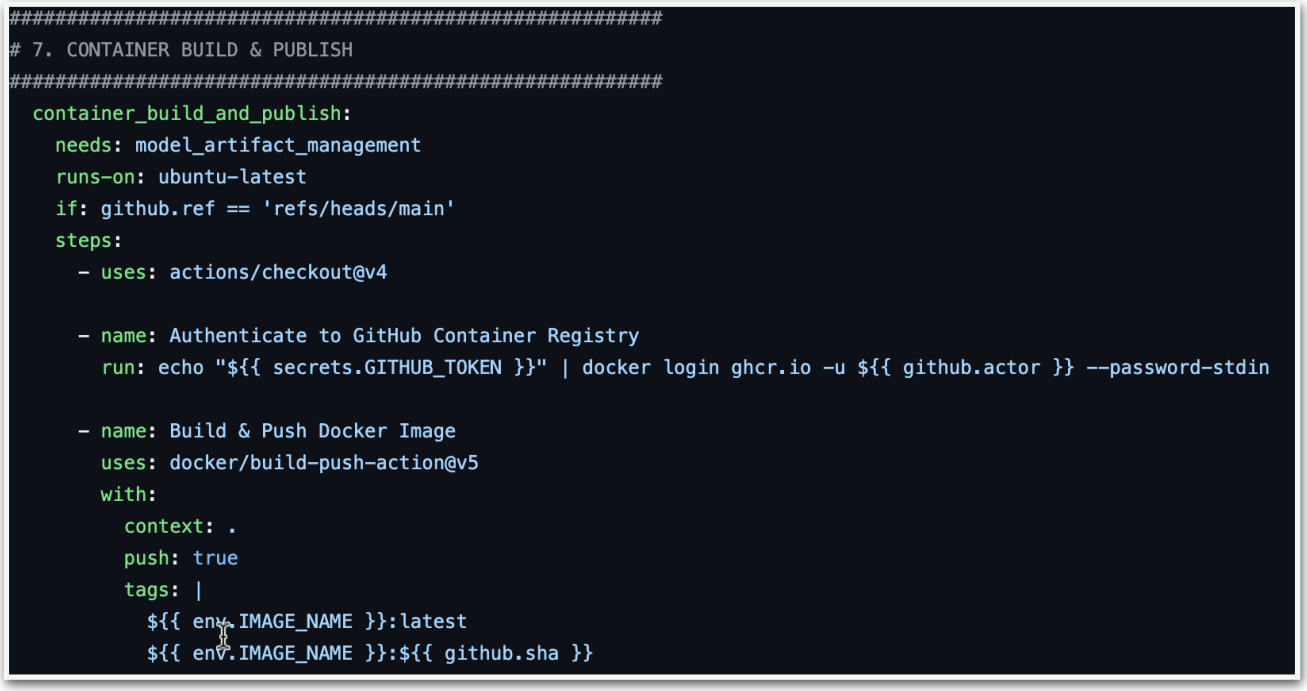
- **https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/Dockerfile**
- **API entry point defined inside the container**
  - **https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/src/serving/app.py**
- Model artifacts loaded at runtime from container filesystem

Container Build & Run Proof

```
docker build -t heart-disease-api .
docker run -p 8000:8000 heart-disease-api
```

Script:
https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/scripts/run_docker.sh

Action YAML File (GITHUB ACTIONS) :
https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/.github/workflows/mlops-pipeline.yml



```yaml
##########################################################
# 7. CONTAINER BUILD & PUBLISH
##########################################################
  container_build_and_publish:
    needs: model_artifact_management
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4

      - name: Authenticate to GitHub Container Registry
        run: echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.actor }} --password-stdin

      - name: Build & Push Docker Image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ${{ env.IMAGE_NAME }}:latest
            ${{ env.IMAGE_NAME }}:${{ github.sha }}
```

```yaml
##########################################################
# 8. DEPLOYMENT & Validation
##########################################################
  deployment-and-validation:
    needs: container_build_and_publish
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4

      - name: Authenticate to GitHub Container Registry
        run: echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.actor }} --password-stdin

      - name: Pull Docker Image
        run: docker pull ${{ env.IMAGE_NAME }}:latest

      - name: Run API Container
        run: |
          docker run -d --rm \
            -p ${{ env.API_PORT }}:${{ env.API_PORT }} \
            --name heart-api \
            ${{ env.IMAGE_NAME }}:latest
```

Script to be executed : (For local Docker, Kubernetes and Ingress setup:

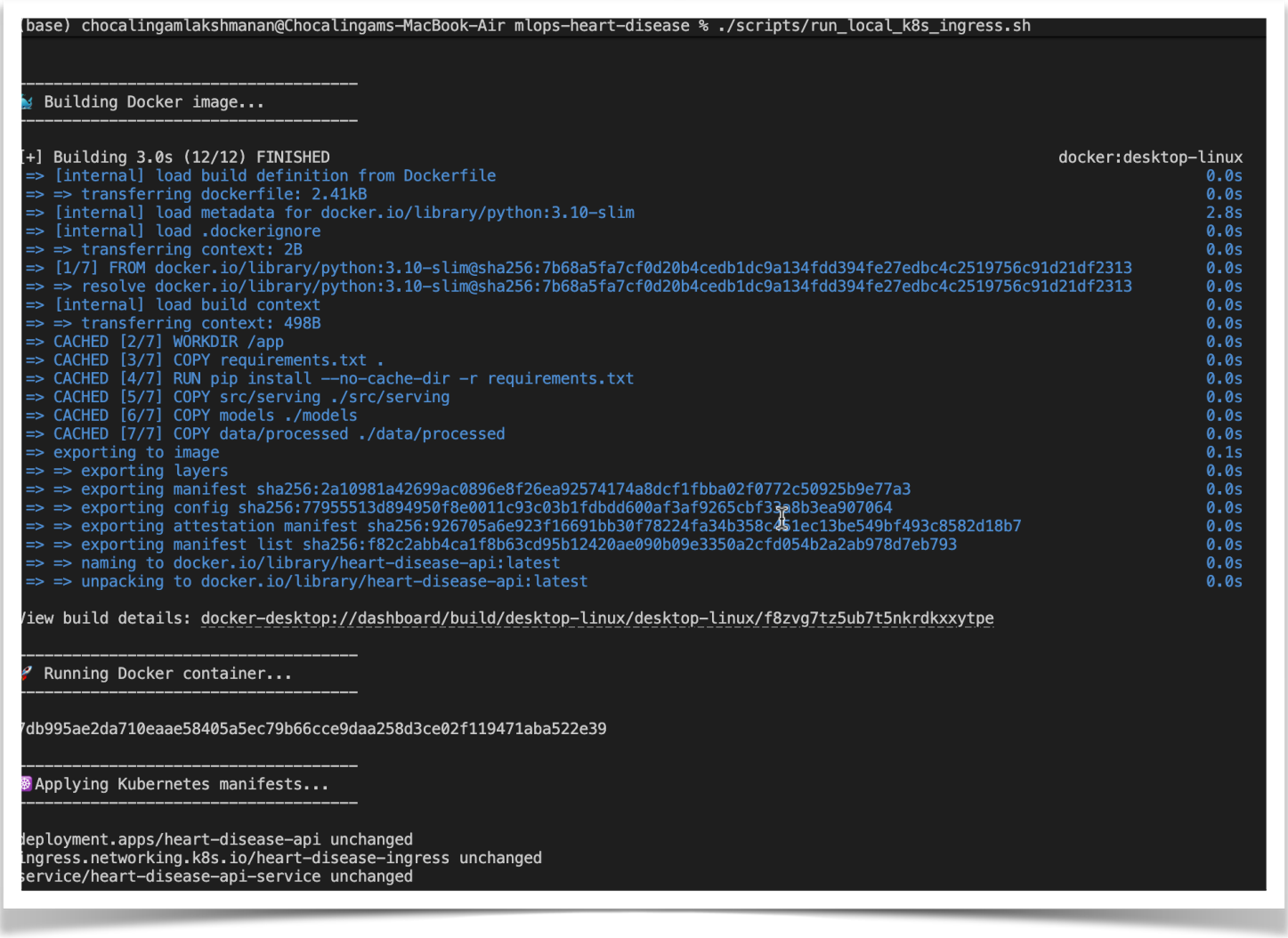
https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/scripts/run_local_k8s_ingress.sh

run_scripts.sh | Dockerfile | CI-CD-pipeline-github-actions.png | pipeline-runtime-usage.png

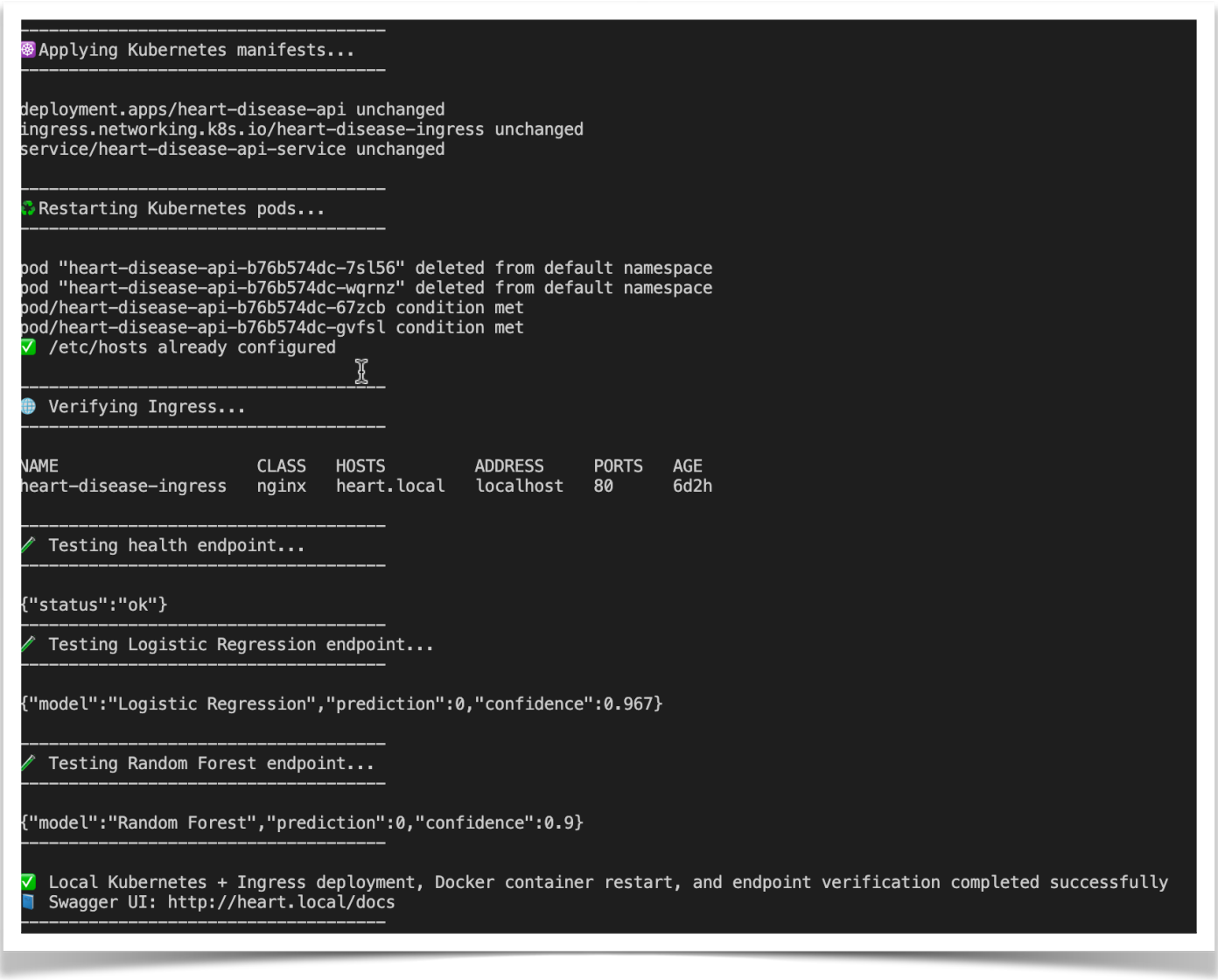
mlops-heart-disease > Dockerfile

```
26  # ------------------------------------------------------------
27  RUN pip install --no-cache-dir -r requirements.txt
28
29  # ------------------------------------------------------------
30  # Copy application source code (FastAPI app)
31  # Contains /predict endpoints
32  # ------------------------------------------------------------
33  COPY src/serving ./src/serving
34
35  # ------------------------------------------------------------
36  # Copy trained ML model artifacts
37  # These .pkl files are created during training phase
38  # ------------------------------------------------------------
39  COPY models ./models
40
41  # ------------------------------------------------------------
42  # Copy preprocessing artifacts (scaler)
43  # Ensures the same preprocessing used during training
44  # is applied during inference
45  # ------------------------------------------------------------
46  COPY data/processed ./data/processed
47
48  # ------------------------------------------------------------
49  # Expose port 8000
50  # This is the port on which FastAPI/Uvicorn runs
51  # ------------------------------------------------------------
52  EXPOSE 8000
53
54  # ------------------------------------------------------------
55  # Start the FastAPI application using Uvicorn
56  # --host 0.0.0.0 allows access from outside the container
57  # --port 8000 matches the exposed port
58  # ------------------------------------------------------------
59  CMD ["uvicorn", "src.serving.app:app", "--host", "0.0.0.0", "--port", "8000"]
60
```

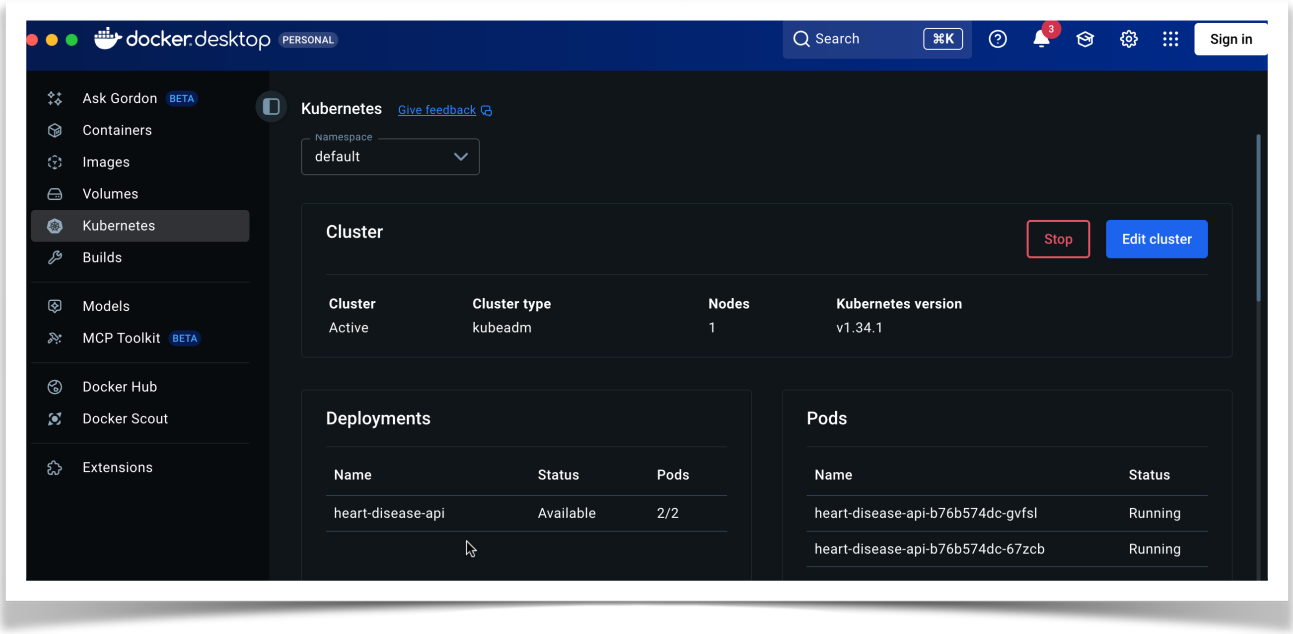
Build and Run Docker Container:



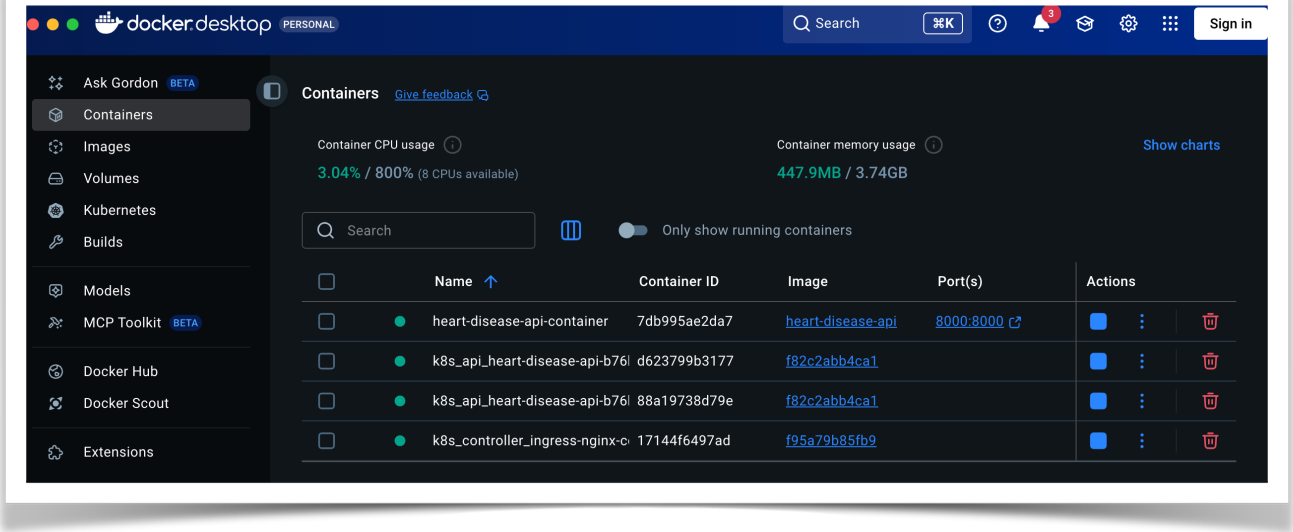
Kubernetes and Ingress Setup and Validation of API Endpoints:

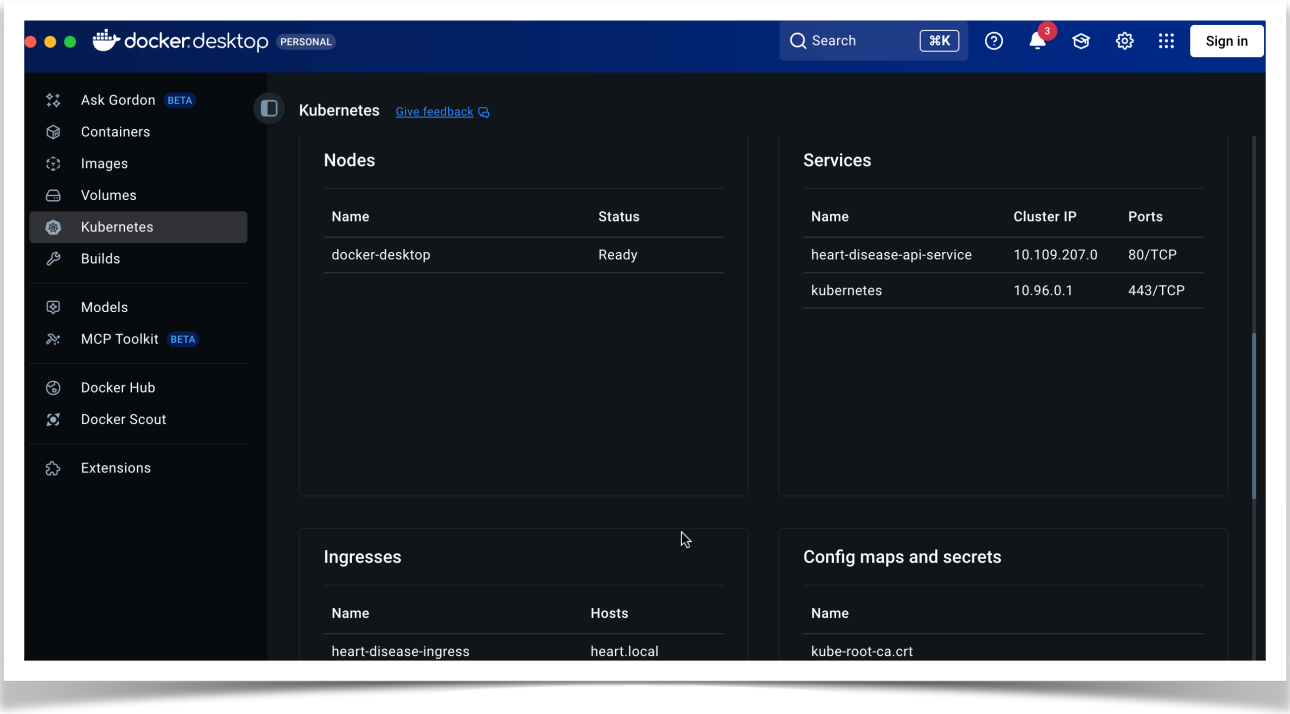


Docker Desktop: (showing Containers)



Docker Desktop: (Showing Kubernetes Cluster with POD, Service, Deployment, Node, Ingress, Config Maps and Secrets:





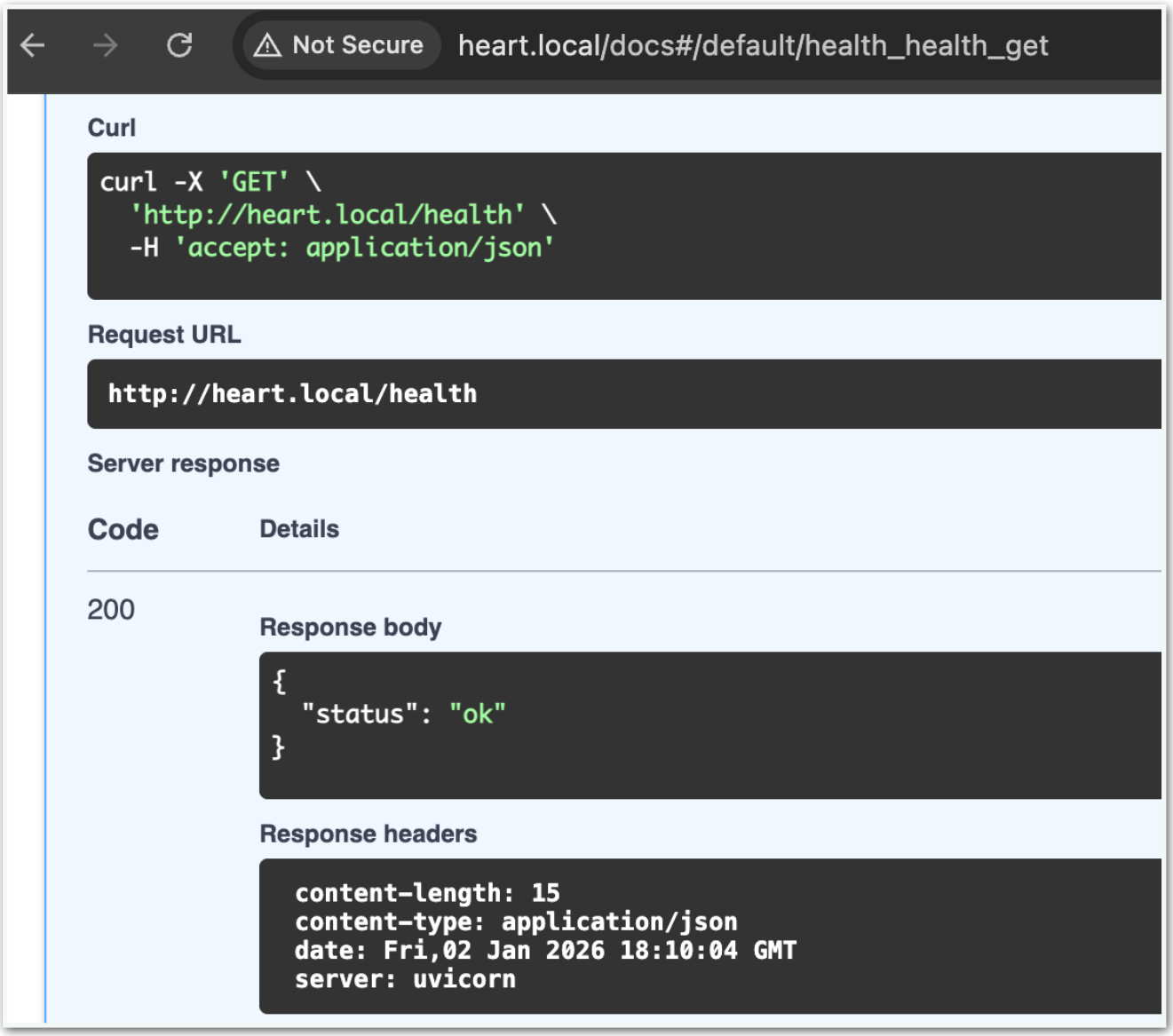
Validation

- API responds correctly to inference requests.
- Model loads successfully inside the container without external dependencies.
- Confirms **environment isolation**, portability, and deployment readiness.

🔷 Step 1: Health Check Validation

Confirms application liveness.

- Calls http://heart.local/health endpoint
- Verifies API startup success


Not Secure
heart.local/docs#/default/health_health_get
Curl
curl -X 'GET' \
'http://heart.local/health' \
-H 'accept: application/json'
Request URL
http://heart.local/health
Server response
Code
Details
200
Response body
{
"status": "ok"
}
Response headers
content-length: 15
content-type: application/json
date: Fri,02 Jan 2026 18:10:04 GMT
server: uvicorn

🔷 Step 2: Model Inference Validation

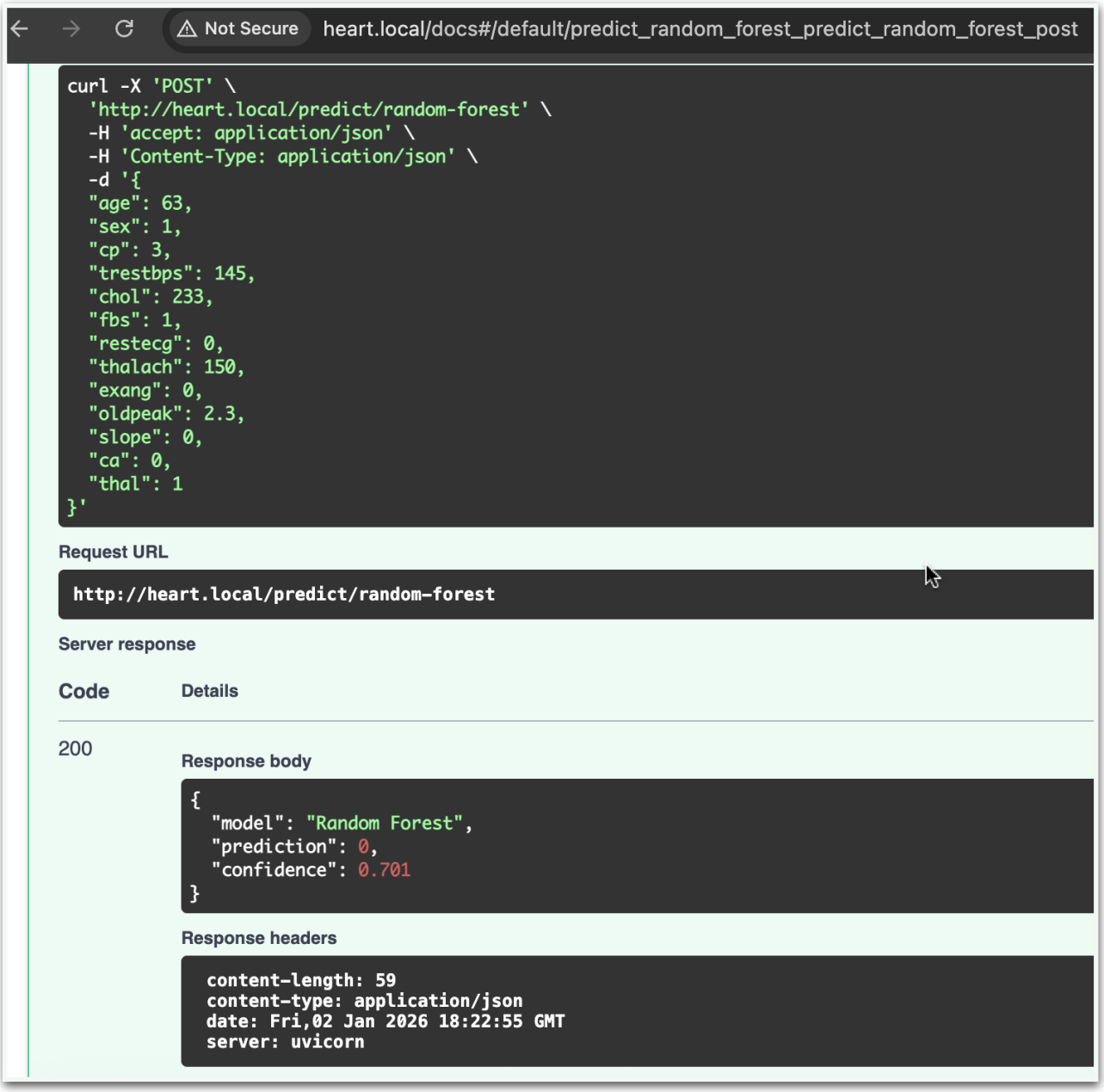
Validates ML endpoints.

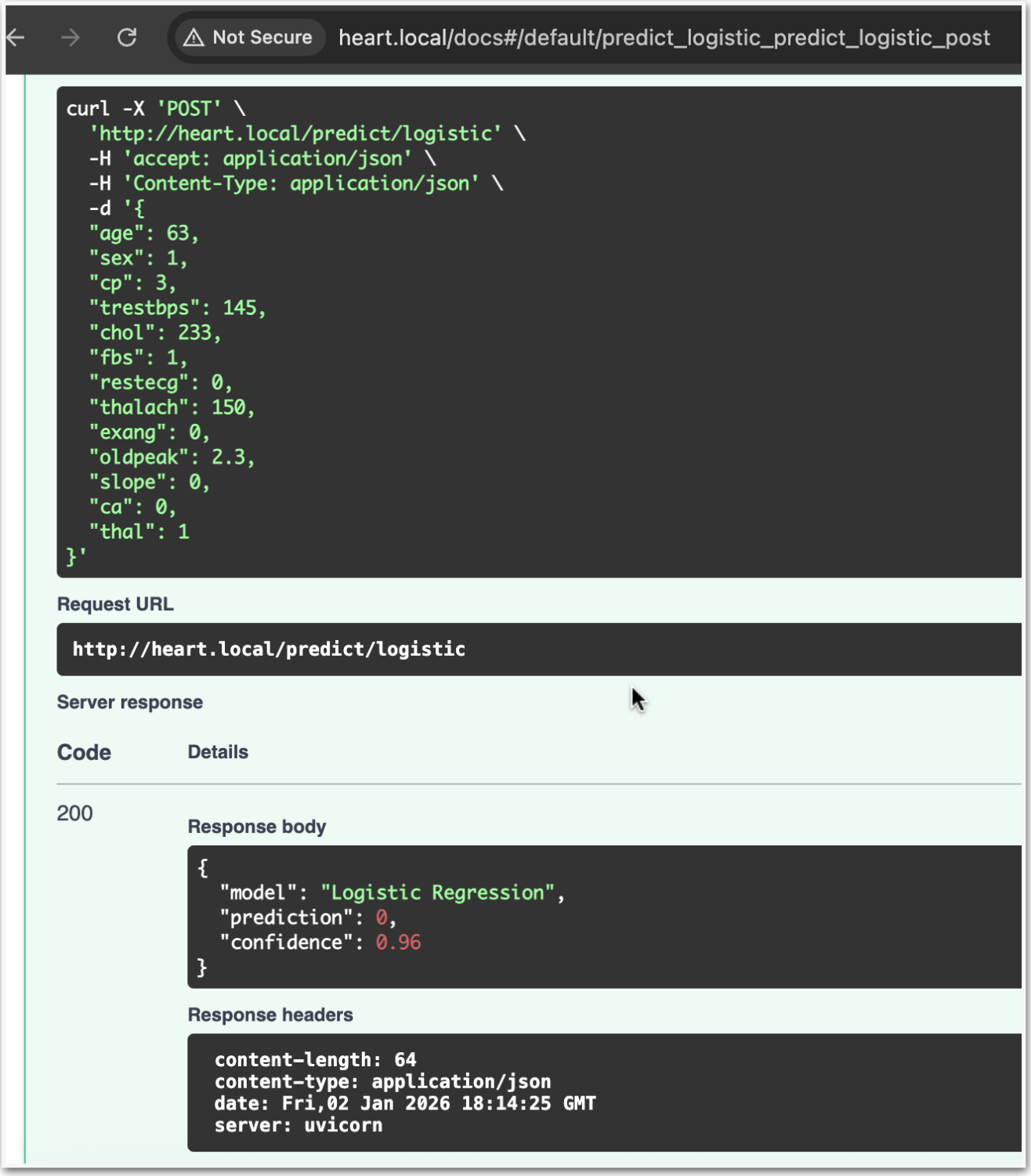
- Tests **Logistic Regression** prediction endpoint

  http://heart.local/predict/logistic

- Tests **Random Forest** prediction endpoint

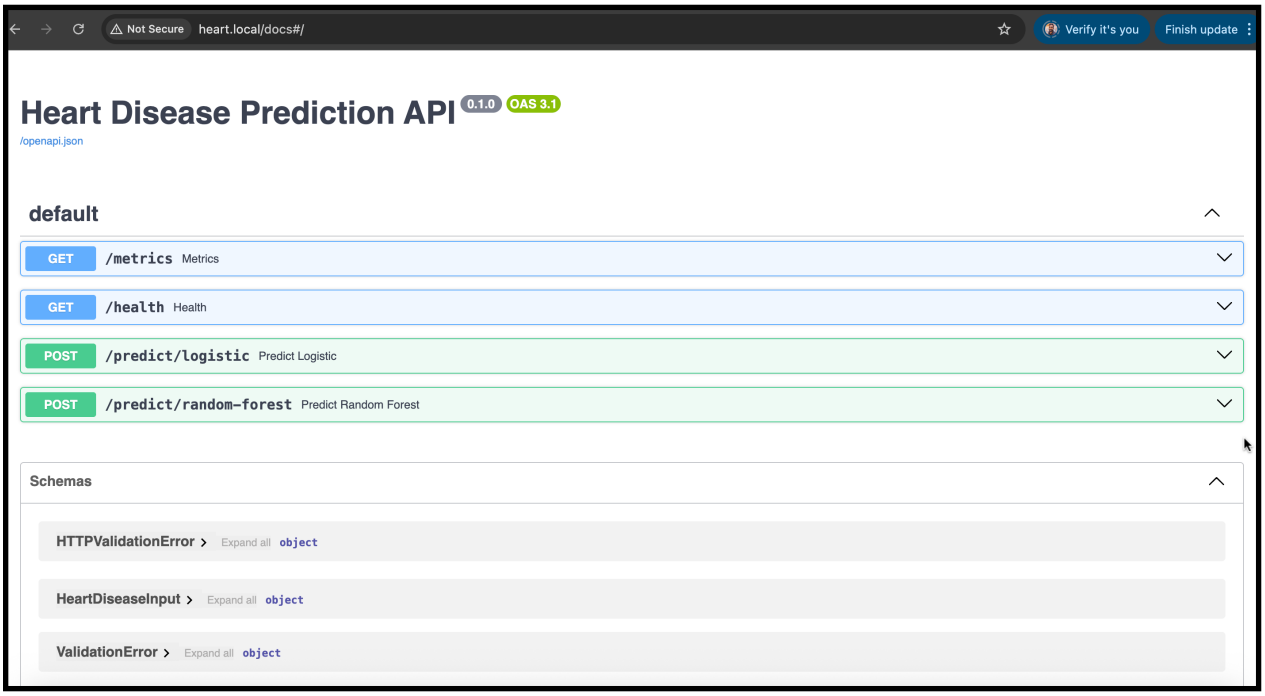
  http://heart.local/predict/random-forest




Not Secure
heart.local/docs#/default/predict_logistic_predict_logistic_post
curl -X 'POST' \
'http://heart.local/predict/logistic' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"age": 63,
"sex": 1,
"cp": 3,
"trestbps": 145,
"chol": 233,
"fbs": 1,
"restecg": 0,
"thalach": 150,
"exang": 0,
"oldpeak": 2.3,
"slope": 0,
"ca": 0,
"thal": 1
}'
Request URL
http://heart.local/predict/logistic
Server response
Code
Details
200
Response body
{
"model": "Logistic Regression",
"prediction": 0,
"confidence": 0.96
}
Response headers
content-length: 64
content-type: application/json
date: Fri,02 Jan 2026 18:14:25 GMT
server: uvicorn

🔷 Step 3: API Access Confirmation

- Swagger UI available at: **http://heart.local/docs**



3. Pipeline Failure on Errors with Clear Logs

Requirement

Pipeline must fail on code or test errors and give clear logs.

Implementation

- CI/CD pipeline is implemented using **GitHub Actions**.
- Pipeline stages include:
  - Dependency installation
  - Linting (code quality checks)

    - Unit testing (`pytest`)
    - Docker build and validation
- Any failure in linting, tests, or build **immediately fails the pipeline**.

Evidence in Codebase

- https://github.com/2024ab05275-chocs/mlops-heart-disease/blob/ci-mlops-pipeline/.github/workflows/mlops-pipeline.yml
- Explicit `set -e` behavior through GitHub Actions step execution
- Logging enabled via:
    - Python `logging` module
    - Docker logs
    - CI job logs visible in GitHub Actions UI

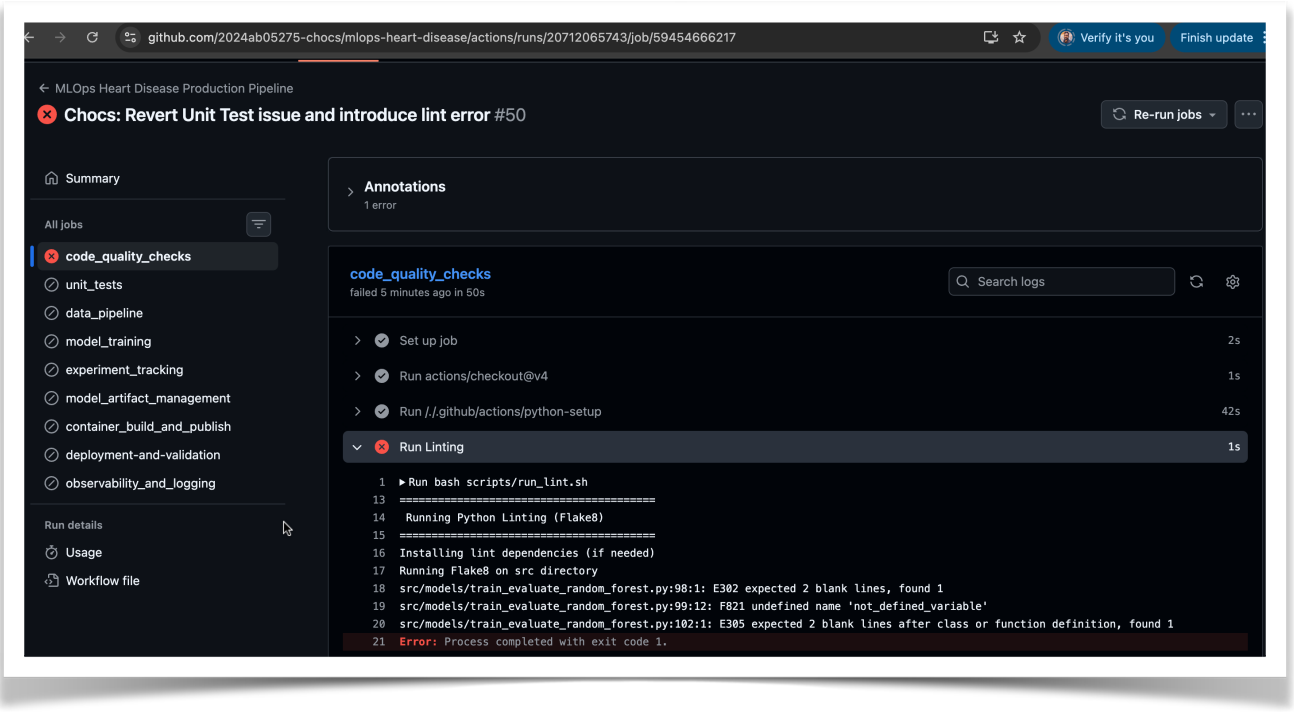
Example Failure Scenarios

Failure Type	Pipeline Behavior
Linting error	Job fails with file/line details
Unit test failure	Pipeline stops with pytest traceback

Linting Error Failure:

Pipeline Showing Lint failure:

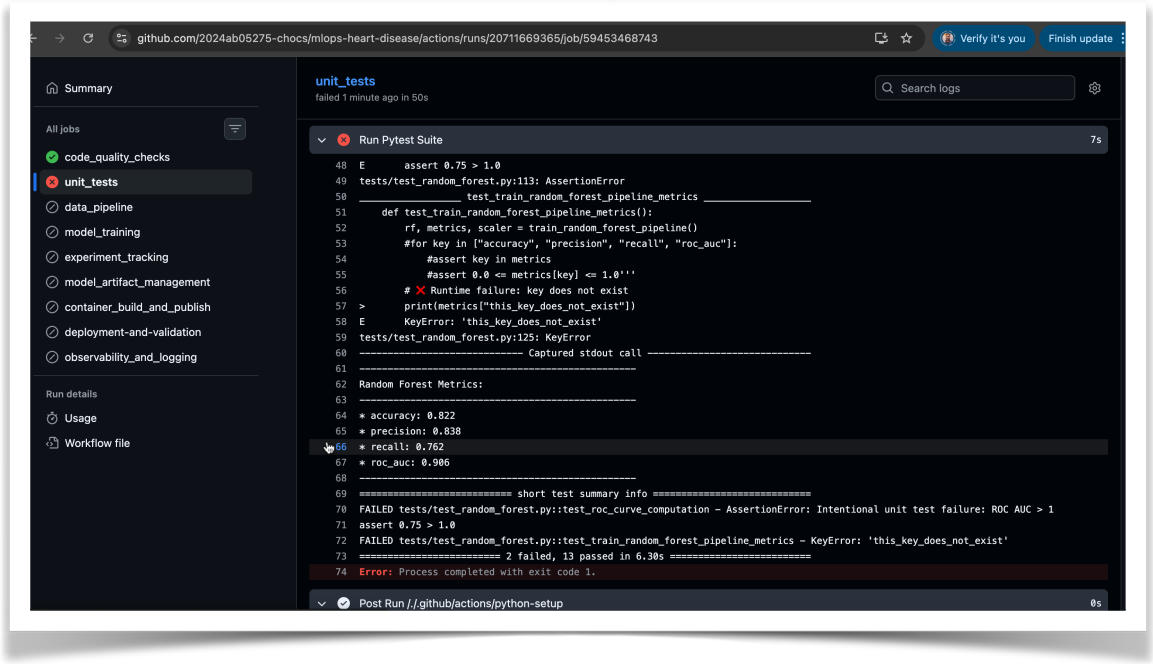
Pipeline: https://github.com/2024ab05275-chocs/mlops-heart-disease/actions/runs/20712065743/job/59454666217



Unit Test Case

Pipeline Showing Unit Test failure:

Pipeline: https://github.com/2024ab05275-chocs/mlops-heart-disease/actions/runs/20711669365/job/59453468743



Observability

- Logs are surfaced in:
    - GitHub Actions job console
    - Docker container logs (`docker logs <container_id>`)
- Ensures rapid debugging and traceability